Next.js Mastery Series

Topic 1: Introduction to Next.js

Beginner → Intermediate → Expert | Interview Q&A with Answers

Akshay Chavhan | Target: Syngenta India + All Companies | April 2026

■ BEGINNER LEVEL

What is Next.js?

Next.js is a **React framework** built on top of Node.js that provides server-side rendering, file-based routing, API routes, and built-in optimizations. Maintained by **Vercel**, it turns React into a true full-stack solution.

■ Key Insight: React = UI Library only. Next.js = Full-Stack Framework built ON TOP of React.

React vs Next.js — Feature Comparison

Feature	React (CRA)	Next.js
Rendering	Client-Side Only	SSR, SSG, ISR, CSR — all supported
Routing	Manual (React Router)	File-based (automatic — no setup)
API Routes	No (needs separate backend)	Built-in /api routes
SEO	Poor (empty HTML shell)	Excellent (pre-rendered HTML)
Image Optimization	Not built-in	next/image component built-in
Full-Stack Capable	No	Yes
Performance	Moderate	Optimized out-of-the-box

Project Structure — With and Without src/ Folder

Both structures are **100% valid**. The **src/** folder is just a personal/team preference chosen during **create-next-app** setup. Next.js supports both. For enterprise/professional projects like Syngenta, using src/ is recommended to cleanly separate source code from config files.

WITHOUT src/ (smaller projects)

WITH src/ (recommended — enterprise)

```
my-app/
■■■ app/
■   ■■■ layout.tsx
■   ■■■ page.tsx
■■■ components/
■■■ public/
■■■ next.config.js
```

```
my-app/
■■■ src/
■   ■■■ app/
■   ■   ■■■ layout.tsx
■   ■   ■■■ page.tsx
■   ■■■ components/
■■■ public/
■■■ next.config.js
```

■ Tip: During npx create-next-app, answer "Yes" to "Would you like to use src/ directory?" for cleaner project structure.

Key Files Explained

app/layout.tsx — Root wrapper (persists across navigation):

```
export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang='en'>
      <body>{children}</body>   {/* All pages render here */}
    </html>
  );
}
```

app/page.tsx — Home page component at route /

```
export default function HomePage() {
  return <h1>Welcome to Next.js!</h1>;
}
```

■ INTERMEDIATE LEVEL

Pages Router vs App Router

	Pages Router (Legacy)	App Router (Modern — v13+)
Directory	/pages folder	/app folder
Default Component	Client Component	Server Component
Data Fetching	getServerSideProps / getStaticProps	async/await directly in component
Layouts	_app.tsx (single global layout)	layout.tsx (nested per route)
Streaming	Not supported	Built-in with Suspense

Recommended ?	Legacy projects only	YES — current standard

The 4 Rendering Strategies — Both Routers Explained

This is the **most important concept** in Next.js. The same 4 strategies exist in both routers — only the **syntax/mechanism** differs. The strategy depends on WHERE the component runs and HOW data is fetched — **not just on using fetch()**. Axios, prisma, any async call works too.

Strategy	App Router Syntax	Pages Router Syntax	Use When
CSR (Client)	use client + hooks	useEffect (no getServerSideProps)	Interactive UI, logged-in dashboards
SSR (Server)	fetch + cache:no-store	export getServerSideProps()	Real-time, user-specific, SEO pages
SSG (Static)	fetch (default, no cache opt)	export getStaticProps()	Blogs, docs, rarely-changing content
ISR (Incremental)	fetch + next:{revalidate:N}	getStaticProps + revalidate:N	Crop prices, news, product listings

Mental Decision Tree — Which Strategy to Use?

```
Is the page interactive (user clicks, state, hooks)?
  ■■■ YES  → CSR  ('use client' + useState/useEffect)
  ■■■ NO   → Does it need FRESH data on every request?
      ■■■ YES  → SSR  (cache: 'no-store' OR getServerSideProps)
      ■■■ NO   → Does data change occasionally?
          ■■■ YES  → ISR  (revalidate: N seconds)
          ■■■ NO   → SSG  (default / getStaticProps)
```

Deep Dive: Each Strategy with Analogy + Code

■ SSG Frozen Pizza	■ SSR Fresh Per Order	■ ISR Batch Refresh	■ CSR You Cook It
Made once at build, served instantly to everyone from CDN.	Made fresh for each visitor. Always up-to-date but slower.	Made once, auto-refreshed every N seconds in background.	Browser downloads JS and renders page itself. Empty HTML first.

1. CSR — Client Side Rendering

Rendered in the **browser**. Page is empty HTML until JS downloads and runs. Good for interactive UIs but bad for SEO.

```
// App Router — CSR
'use client';
import { useEffect, useState } from 'react';

export default function Dashboard() {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch('/api/user').then(r => r.json()).then(setUser);
  }, []);

  if (!user) return <p>Loading...</p>;
  return <h1>Welcome {user.name}</h1>;
}

// Good for: Logged-in dashboards, interactive charts, user-specific UIs
// Bad for:  Public SEO pages, slow connections (blank screen on load)
```

2. SSG — Static Site Generation

HTML is generated **once at build time** and served from CDN. Fastest possible delivery. Data is baked-in. Both App Router and Pages Router shown.

```
// App Router — SSG (default behavior, no cache option needed)
async function BlogPage() {
  const res = await fetch('https://api.example.com/posts');
  // No cache option = Next.js caches this response at BUILD TIME
  const posts = await res.json();
  return <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}

// Pages Router — SSG with getStaticProps
export async function getStaticProps() {
  const res = await fetch('https://api.example.com/posts');
  const posts = await res.json();
  return { props: { posts } };  // No revalidate = pure SSG
}
export default function BlogPage({ posts }) {
  return <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}
```

3. SSR — Server Side Rendering

HTML is generated **on every request** on the server. Always fresh data. Good for SEO + dynamic content. Higher server load than SSG.

```
// App Router — SSR (cache: 'no-store' forces fresh fetch every time)
async function StockPage() {
  const res = await fetch('https://api.example.com/stocks', {
    cache: 'no-store',  // Never cache — always fetch fresh from server
  });
  const stocks = await res.json();
  return <div>NIFTY: {stocks.nifty}</div>;
}

// Pages Router — SSR with getServerSideProps
export async function getServerSideProps(context) {
  const res = await fetch('https://api.example.com/stocks');
  const stocks = await res.json();
  return { props: { stocks } };
}
export default function StockPage({ stocks }) {
  return <div>NIFTY: {stocks.nifty}</div>;
}
```

4. ISR — Incremental Static Regeneration

Best of both worlds: **SSG speed + periodic freshness**. Page is statically generated but Next.js regenerates it in the background after the revalidate time expires. Perfect for Syngenta crop price data!

```
// App Router — ISR with next: { revalidate: N }
async function CropPricePage() {
  const res = await fetch('https://api.syngenta.com/crop-prices', {
    next: { revalidate: 3600 },  // Regenerate every 1 hour in background
  });
  const prices = await res.json();
  return <div>Wheat: Rs.{prices.wheat}/kg</div>;
}

// Pages Router — ISR with revalidate in getStaticProps
export async function getStaticProps() {
  const res = await fetch('https://api.syngenta.com/crop-prices');
  const prices = await res.json();
  return {
    props: { prices },
    revalidate: 3600,  // Regenerate page every 1 hour
  };
}
```

Using Axios Instead of fetch — Does Strategy Change?

■ Important: Rendering strategy does NOT depend on fetch alone. It depends on WHERE your component runs (server/browser) and HOW you export/declare it. Axios works perfectly in Next.js!

The key difference: **fetch()** in Next.js has special caching built-in (cache options control SSG/SSR/ISR). **Axios** has no Next.js cache integration — so rendering strategy is determined by the component type/export, not axios config.

```
// AXIOS — SSR (App Router Server Component — default = SSR)
import axios from 'axios';

async function ProductPage() {
  // Runs on SERVER. No cache control = treated as SSR (fresh each request)
  const { data } = await axios.get('https://api.example.com/product');
  return <div>{data.name}</div>;
}

// AXIOS — CSR (Client Component)
'use client';
import { useEffect, useState } from 'react';
import axios from 'axios';

function ProductClient() {
  const [data, setData] = useState(null);
  useEffect(() => {
    axios.get('/api/product').then(res => setData(res.data));
  }, []);
  return <div>{data?.name}</div>;
}

// AXIOS — SSG (Pages Router getStaticProps)
export async function getStaticProps() {
  const { data } = await axios.get('https://api.example.com/posts');
  return { props: { posts: data } };  // Build-time only = SSG
}

// AXIOS — ISR (Pages Router getStaticProps + revalidate)
export async function getStaticProps() {
  const { data } = await axios.get('https://api.example.com/posts');
  return { props: { posts: data }, revalidate: 60 };  // ISR
}
```

HTTP Method	SSR	SSG	ISR	CSR
fetch()	cache:no-store	default (no cache opt)	next:{revalidate:N}	use client + useEffect
axios	Server Component (async fn)	getStaticProps	getStaticProps+revalidate	use client + useEffect
prisma/DB	Server Component (async fn)	getStaticProps	getStaticProps+revalidate	Via API route only

■ EXPERT LEVEL

Server Components vs Client Components

■ Golden Rule: Default to Server Components. Add "use client" ONLY when you need React hooks, event handlers, or browser APIs.

```
// SERVER COMPONENT — default in App Router (no directive needed)
// CAN:  fetch data, access DB with Prisma, read server env secrets
// CANNOT: useState, useEffect, onClick, window, document
async function UserProfile({ userId }: { userId: string }) {
  const user = await prisma.user.findUnique({ where: { id: userId } });
  return <div>Hello, {user?.name}</div>;  // Zero JS sent to browser
}

// CLIENT COMPONENT — must have 'use client' at very top of file
// CAN:  useState, useEffect, onClick, window, document, browser APIs
// CANNOT: be async, directly access DB, read secret env vars
'use client';
import { useState } from 'react';

function LikeButton({ postId }: { postId: string }) {
  const [liked, setLiked] = useState(false);
  return (
    <button onClick={() => setLiked(!liked)}>
      {liked ? 'Liked!' : 'Like'}
    </button>
  );
}
```

How Next.js Works Internally

```
Browser Request → Next.js Server
      ↓
  Route matched via file-system (app/products/page.tsx → /products)
      ↓
  Server Component tree rendered (async, DB access happens here)
      ↓
  RSC Payload generated (binary format — NOT raw HTML)
      ↓
  HTML streamed to browser in chunks (fast first byte)
      ↓
  Client Components hydrated (JS event listeners attached)
      ↓
  Page fully interactive
```

Hydration Explained

```
// Without Next.js (pure React/CRA):
Server sends: <div id='root'></div>   // Empty HTML!
Browser: downloads JS (slow) → renders → interactive
= Blank white screen + poor SEO + slow LCP

// With Next.js SSR/SSG:
Server sends: <div><h1>Hello Akshay</h1><p>Crop data...</p></div>
Browser: shows HTML IMMEDIATELY (fast paint, great SEO)
React then 'hydrates' → attaches event listeners to existing DOM
= Fast first paint + perfect SEO + full interactivity
```

Environment Variables — Server vs Browser

```
# .env.local

NEXT_PUBLIC_API_URL=https://api.example.com
# ↑ NEXT_PUBLIC_ prefix = exposed to browser (client-safe only!)

DATABASE_URL=postgresql://...
SECRET_KEY=my-super-secret
# ↑ No prefix = server only. NEVER sent to browser. SAFE.

# In code:
const apiUrl = process.env.NEXT_PUBLIC_API_URL;  // Works in browser
const dbUrl  = process.env.DATABASE_URL;         // Server only
```

Critical Gotchas

■■ Gotcha 1: 'use client' propagates downward. If a parent is a Client Component, ALL its children become Client Components too — even if they don't need to be. Push 'use client' as deep (close to leaf) as possible.

■■ Gotcha 2: Server Components cannot use React hooks. Adding useState/useEffect in a Server Component throws an error at build/runtime.

■■ Gotcha 3: Fetching with axios in App Router Server Components = SSR by default (no ISR/SSG caching control). Use fetch() when you need Next.js cache control. Use axios for non-cached server requests or CSR.

■■ Gotcha 4: next.config.js changes require server restart. Changes to this file do not hot-reload — you must stop and restart npm run dev.

■ INTERVIEW Q&A WITH DETAILED ANSWERS

Beginner Level

BEGINNER

Q1. What is Next.js and how is it different from React?

Answer: Next.js is a React framework that adds SSR, SSG, ISR, file-based routing, and built-in API routes on top of React. React is just a UI library — it only handles the view layer. Next.js makes React full-stack: you can write both frontend and backend (API routes) in one project.

BEGINNER

Q2. What are the main features of Next.js?

Answer: File-based routing (folders become routes automatically), 4 rendering strategies (CSR/SSR/SSG/ISR), built-in API routes (/api folder), Server and Client Components (App Router), image optimization via next/image, font optimization via next/font, TypeScript support out-of-the-box, and easy Vercel deployment.

BEGINNER

Q3. What is the difference between pages/ and app/ directory?

Answer: pages/ is the legacy Pages Router — components are client-side by default, data fetching uses getServerSideProps/getStaticProps. app/ is the modern App Router (Next.js v13+) — components are Server Components by default, data fetching uses async/await directly, and supports nested layouts. For new projects, always use App Router.

BEGINNER

Q4. Some projects don't have a src/ folder. Is that a problem?

Answer: No, both structures are valid. Whether you use src/ is chosen during create-next-app setup. Without src/, the app/ folder sits at the project root. With src/, it sits inside src/. Both work identically. Use src/ for larger/enterprise projects to separate source code from config files.

Intermediate Level

INTERMEDIATE

Q5. Explain the 4 rendering strategies in Next.js.

Answer: CSR: Rendered in browser — 'use client' + hooks. Good for interactive dashboards. | SSR: Rendered on server per request — cache:'no-store'. Good for real-time, SEO pages. | SSG: Rendered at build time once — default fetch. Good for blogs, docs. | ISR: SSG + background refresh every N seconds — next:{revalidate:N}. Good for semi-static content like crop prices.

INTERMEDIATE

Q6. Do rendering strategies only work with fetch()? What about axios?

Answer: No! Strategy depends on WHERE component runs, not which HTTP library you use. With axios in a Server Component = SSR by default. With axios in a Client Component = CSR. In getStaticProps = SSG. In getStaticProps+revalidate = ISR. fetch() is special because Next.js extends it with cache options for ISR/SSG control. Axios lacks this integration but works fine for SSR and CSR.

INTERMEDIATE

Q7. What is the difference between Server and Client Components?

Answer: Server Components (default in App Router): run on server, can be async, can access DB/Prisma directly, read secret env vars — but NO hooks, event handlers, or browser APIs. Client Components ('use client'): run in browser, support hooks/events/browser APIs — but cannot be async or access server resources directly. Best practice: push 'use client' as deep as possible.

INTERMEDIATE

Q8. What is hydration and why does it matter?

Answer: Hydration is React attaching JavaScript event listeners to the server-rendered HTML already in the browser. The server sends full HTML (fast first paint, great SEO). Then React 'hydrates' it — connecting the virtual DOM to the real DOM and making it interactive. Without Next.js (plain React), the browser receives empty HTML and waits for JS to download before showing anything.

INTERMEDIATE

Q9. What is the purpose of layout.tsx vs page.tsx?

Answer: page.tsx defines the UI for a specific route — it re-renders on navigation. layout.tsx wraps its child pages and persists across navigation within its route segment — it does NOT re-render. Nested layouts allow different shells for different sections (e.g., /dashboard has a sidebar layout, /marketing has a full-width layout).

Expert / Syngenta-Style Questions

SYNGENTA

Q10. At Syngenta, you have a crop prices dashboard. Data updates every hour. Which rendering strategy and why?

Answer: ISR with revalidate: 3600 (1 hour). Reasoning: Crop prices don't change per-request — so SSR would waste server resources re-rendering on every visit. SSG is too stale if prices change hourly. ISR gives CDN-level speed (fast loads) while Next.js silently regenerates the page in the background every hour. Users always see fresh-enough data without server overhead.

EXPERT

Q11. What is RSC Payload and how does it differ from traditional SSR?

Answer: RSC (React Server Components) Payload is a compact binary format describing the rendered server component tree — it is NOT raw HTML. Traditional SSR generates and sends complete HTML strings. RSC Payload enables selective hydration: only Client Components marked 'use client' get hydrated with JavaScript. Server Components produce zero client JS bundle — reducing bundle size significantly.

EXPERT

Q12. What happens when you add 'use client' to a root layout?

Answer: Catastrophic for performance. Every component in the entire app becomes a Client Component. The entire React tree gets included in the JS bundle sent to the browser — defeating Next.js App Router optimizations entirely. No component can be a Server Component anymore. Best practice: layout.tsx should never have 'use client' — only add it to the smallest interactive leaf components.

EXPERT

Q13. How would you decide between fetch and axios in a Next.js App Router project?

Answer: Use fetch() when you need Next.js cache control: SSG (default), ISR (next:{revalidate:N}), or force SSR (cache:'no-store'). Use axios when: (1) you need interceptors for auth headers/error handling, (2) you're doing CSR in Client Components, (3) you want cleaner syntax for POST/PUT with request bodies, (4) you're calling APIs from Server Components where caching isn't needed. For data fetching with caching strategy, fetch() is the Next.js-native choice.